

Volume 14 – Testing, Quality Assurance & Validation Strategy

AI Quant Trading Platform

Version: 1.0

Document Type: Testing & Quality Assurance

Classification: Enterprise QA Framework

1. Purpose

This document defines the testing methodology, quality assurance processes, validation requirements, acceptance criteria, and release procedures for the AI Quant Trading Platform.

The objective is to ensure the platform remains reliable, secure, performant, and maintainable while reducing the risk of defects in production.

2. QA Objectives

The QA process shall ensure:

- Stable releases
 - Accurate calculations
 - Reliable automation
 - Secure authentication
 - Correct trade execution
 - Consistent API behavior
 - Performance under load
 - Safe deployment
-

3. Testing Pyramid

```
Manual Exploratory
-----
End-to-End Automation
-----
Integration Testing
```

Unit Testing

Most automated tests should exist at the lower layers.

4. Testing Categories

Unit Testing

Scope

- Services
- Utilities
- Business Rules
- Validators
- Helpers
- AI Score Calculations
- Risk Rules

Target Coverage

Minimum 85%

Integration Testing

Verify communication between:

- Django ↔ PostgreSQL
 - Django ↔ Redis
 - Django ↔ Celery
 - Django ↔ Exchange Adapter
 - API ↔ AI Engine
 - Strategy ↔ Risk Engine
-

API Testing

Validate:

- Authentication
- Authorization
- Request Validation
- Response Schema
- Error Handling
- Pagination
- Filtering
- Rate Limits

Every public endpoint must have automated tests.

Database Testing

Verify

- Constraints
 - Foreign Keys
 - Indexes
 - Transactions
 - Rollback
 - Migrations
-

UI Testing

Test

- Navigation
 - Responsive Design
 - Forms
 - Charts
 - Tables
 - Accessibility
 - Theme Switching
-

End-to-End Testing

Typical flow:

Register

↓

Connect Exchange

↓

Configure Strategy

↓

Run Paper Trading

↓

Review Results

↓

Enable Live Trading

↓

Receive Notifications

5. Trading Validation

Every strategy must be validated using:

- Historical Backtesting
- Paper Trading
- Controlled Live Testing

Before production release.

6. AI Validation

Validate

- Score Consistency
- Recommendation Logic
- Confidence Values
- Market Regime Classification
- Explainability Output

The AI should produce deterministic results when given identical inputs.

7. Machine Learning Validation

Measure

- Precision
- Recall
- F1 Score
- ROC-AUC
- MAE
- RMSE
- Drift Detection

Production models require documented validation.

8. Exchange Testing

Verify

- API Authentication
- Balance Sync
- Order Placement
- Order Cancellation
- WebSocket Streaming
- Reconnection Logic
- Rate Limit Handling

Use sandbox or paper environments whenever available.

9. Security Testing

Perform

- Authentication Tests
 - Authorization Tests
 - Session Tests
 - Input Validation
 - Rate Limiting
 - Dependency Scans
 - Penetration Testing
-

10. Performance Testing

Measure

- API Response Time
 - Database Queries
 - Worker Throughput
 - Scanner Speed
 - WebSocket Latency
 - Order Processing Time
-

11. Load Testing

Simulate

- 1,000 Users
- 5,000 Users
- 10,000 Users

- Multiple Exchanges
- Large Watchlists

Measure

- CPU
 - Memory
 - Queue Length
 - Latency
-

12. Stress Testing

Continue increasing load until:

- System slows
- Resources are exhausted
- Recovery mechanisms activate

Document the platform's limits and recovery behavior.

13. Scalability Testing

Verify:

- Horizontal Scaling
 - Worker Scaling
 - Redis Scaling
 - Database Scaling
-

14. Reliability Testing

Test

- Exchange Failure
- Redis Failure
- Database Failure
- Worker Failure
- Network Interruption

The platform should recover gracefully where possible.

15. Regression Testing

Automatically execute regression tests for:

- Every pull request
 - Every release candidate
 - Every production deployment
-

16. User Acceptance Testing (UAT)

Representative users verify:

- Trading workflows
- Dashboard usability
- Reports
- Notifications
- Configuration

Document issues and approvals.

17. Test Data Management

Maintain

- Synthetic Market Data
- Historical Datasets
- Mock Exchange Responses
- Test Accounts

Avoid using production credentials in testing.

18. Continuous Testing

CI pipeline executes:

- Unit Tests
- Integration Tests
- API Tests
- Linting
- Security Scans

Deployment is blocked if required checks fail.

19. Test Automation

Recommended Tools

- pytest
- pytest-django
- Playwright
- Postman/Newman
- Locust
- k6

Tool selection may evolve over time.

20. Bug Management

Every defect records:

- ID
 - Severity
 - Priority
 - Environment
 - Steps to Reproduce
 - Expected Result
 - Actual Result
 - Status
-

21. Release Criteria

A release is approved only if:

- Critical defects are resolved.
 - Required automated tests pass.
 - Security checks pass.
 - Database migrations are verified.
 - Rollback plan is documented.
 - Monitoring and alerting are enabled.
-

22. Production Validation

After deployment verify:

- Login
- Exchange Connectivity
- Scanner

- AI Engine
 - Strategy Engine
 - Risk Engine
 - Order Execution
 - Notifications
 - Reports
-

23. Success Metrics

Track

- Test Coverage
- Defect Density
- Escaped Defects
- Mean Time to Detect
- Mean Time to Resolve
- Release Success Rate

Review these metrics regularly to improve quality processes.

24. Documentation

Maintain

- Test Cases
- Test Plans
- Regression Suites
- UAT Results
- Performance Reports
- Security Reports

Version all QA artifacts.

25. Design Principles

- Test early and continuously.
- Automate repetitive validation.
- Validate trading logic before production use.
- Separate research, paper trading, and live trading environments.
- Every production release must be reproducible and reversible.
- Quality is a shared responsibility across engineering, QA, and operations.